

Tutorial de VHDL no EDA Playground

1. Introdução e configuração do EDA Playground

VHDL descreve hardware, não um programa sequencial: as partes de um circuito funcionam ao mesmo tempo, e o simulador reproduz esse paralelismo. O EDA Playground simula VHDL no navegador, sem instalar ferramentas.

Link: [EDA Playground](#)

Um projeto tem duas partes, cada uma em uma janela do editor:

- Design: o circuito que voce quer construir (entity + architecture).
- Testbench: código usado apenas na simulação; gera clock e estímulos, liga-se ao circuito e confere as saídas. Não vira hardware.

Configuração da sessão

1. Acesse o EDA Playground e entre com uma conta, para poder salvar o projeto.
2. Em Languages & Libraries, escolha VHDL e uma versão (o código deste tutorial compila em VHDL-93 e em VHDL-2008).
3. Em Tools & Simulators, escolha um simulador com suporte a VHDL (o Código foi verificado com o GHDL). No campo *Top entity*, preencha com *tb_contador_mod10*.
4. Cole o código do *testbench* na janela testbench.vhd (esquerda) e o do *design* em design.vhd (direita).
5. Marque *Open EPWave after run* e clique em *Run*.

2. O projeto exemplo: contador modulo 10

O exemplo é um contador decimal: conta 0, 1, 2, ..., 9 e, no pulso seguinte, volta a 0, ou seja, reinicia ao chegar a 10. A contagem avança na borda de subida do *clock*, só quando *en* = '1', e um *reset* assíncrono zera o contador a qualquer momento. Além do valor, o circuito gera um sinal de fim de contagem e o código de um *display* de 7 segmentos.

Constantes são especificadas com aspas ou apóstrofes. "0011", por exemplo, define uma constante de 4 bits, enquanto que '1' e '0' definem constantes de 1 bit.

Porta	Direção	Tipo	Função
clk	in	std_logic	clock; a contagem avança na borda de subida
rst	in	std_logic	reset assíncrono, ativo em '1'; zera a contagem
en	in	std_logic	habilita a contagem
q	out	std_logic_vector(3 downto 0)	valor atual, de 0 a 9

Porta	Direção	Tipo	Função
tc	out	std_logic	'1' quando q = 9 (terminal count)
seg	out	std_logic_vector(6 downto 0)	segmentos g f e d c b a do display, ativos em '1'

Os arquivos do projeto são dois: design.vhd (secoes 3 a 6) e testbench.vhd (seção 7). Cada seção a seguir mostra o trecho do código que ilustra uma construção da linguagem.

3. Entity: a interface do circuito

A entity declara o que o circuito mostra para o mundo exterior (nome, portas, direções e tipos) e nada sobre o que ele faz por dentro. É a "caixa preta" do projeto.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity contador_mod10 is
  port (
    clk  : in  std_logic;           -- clock
    rst  : in  std_logic;           -- reset assíncrono, ativo em '1'
    en   : in  std_logic;           -- habilita a contagem
    q    : out std_logic_vector(3 downto 0); -- valor da contagem
    tc   : out std_logic;           -- '1' quando q = 9
    seg  : out std_logic_vector(6 downto 0) -- display 7 segmentos (g f e d c b a)
  );
end entity contador_mod10;
```

Pontos de atenção:

- *library* e *use* importam pacotes. *std_logic_1164* define *std_logic* (valores '0', '1', 'Z', 'X', entre outros) e *numeric_std* define *unsigned*, *signed* e as operações aritméticas sobre eles.
- Cada porta tem um nome, uma direção (*in*, *out*, *inout* ou *buffer*) e um tipo. Escreva *std_logic* para sinais de um bit e *std_logic_vector* para vetores com vários bits.
- Em (3 downto 0), o bit 3 é o mais significativo e fica à esquerda ao escrever o valor, como em "1001".
- VHDL não diferencia maiúsculas de minúsculas, e comentários começam com --.

4. Architecture: o comportamento do circuito

A *architecture* descreve como a *entity* funciona. Ela tem duas regiões: a parte declarativa, entre *is* e *begin*, onde ficam sinais, constantes e tipos internos, e o corpo, entre *begin* e *end*, onde ficam os comandos que descrevem o circuito.

```
architecture rtl of contador_mod10 is
  -- parte declarativa: sinal interno que guarda a contagem
  signal cnt : unsigned(3 downto 0) := (others => '0');
begin
  -- corpo: comandos concorrentes e processes (secoes 5 e 6)
```

```
...
end architecture rtl;
```

- O nome *rtl* identifica esta implementação. Uma mesma *entity* pode ter várias arquiteturas (por exemplo, *rtl* e *comportamental*), e a escolha é feita na instanciação.
- *cnt* é um sinal *unsigned* porque a contagem usa a operação de soma ($cnt + 1$), que não é definida para o tipo *std_logic_vector*. Como a porta *q* é *std_logic_vector*, o tipo *unsigned* deve ser convertido na saída.
- A contagem fica em um sinal interno, e não direto em *q*, porque em VHDL-93 uma porta *out* não pode ser lida dentro da arquitetura, e o contador precisa ler o próprio valor para ser incrementado.
- (*others => '0'*) preenche todos os bits do vetor com '0', qualquer que seja a largura.

5. Comandos concorrentes

Comandos escritos diretamente no corpo da *architecture*, fora de um *process*, são concorrentes: todos existem ao mesmo tempo, como fios e portas lógicas interligadas, e a ordem em que aparecem no arquivo não importa. Sempre que um sinal de entrada do comando muda, o resultado é recalculado.

```
-- 1. atribuição simples: liga um sinal a outro
q <= std_logic_vector(cnt);

-- 2. atribuição condicional (when/else): equivale a um if/else combinacional
tc <= '1' when cnt = 9 else '0';

-- 3. atribuição selecionada (with/select): equivale a um case combinacional
with cnt select
  seg <= "0111111" when "0000", -- 0
        "0000110" when "0001", -- 1
        "1011011" when "0010", -- 2
        "1001111" when "0011", -- 3
        "1100110" when "0100", -- 4
        "1101101" when "0101", -- 5
        "1111101" when "0110", -- 6
        "0000111" when "0111", -- 7
        "1111111" when "1000", -- 8
        "1101111" when "1001", -- 9
        "0000000" when others; -- valores fora de 0 a 9
```

Comando	Quando usar	Logica gerada
<code>a <= b;</code>	ligar ou renomear sinais	fio
<code>a <= x when c else y;</code>	escolha por condições, com prioridade	multiplexador
<code>with s select a <= ...</code>	escolha por valores de um único sinal	decodificador ou ROM

Regras importantes:

- O `with/select` precisa cobrir todos os valores possíveis; o *when others* garante isso.
- Se você trocar a ordem das três linhas no arquivo, o circuito é o mesmo.
- Um mesmo sinal não deve receber atribuições concorrentes de dois comandos diferentes, pois isso cria um curto-circuito (dois *drivers*).

6. Comandos sequenciais: o process

O *process* é a região em que os comandos são executados sequencialmente, de cima para baixo, como em uma linguagem de programação. É a região onde se descreve lógica sequencial (registradores) e decisões com *if*, *case* e laços. O *process* como um todo é um comando concorrente, executado em paralelo com os demais; só o seu interior é sequencial.

```
p_contagem : process (clk, rst)
begin
  if rst = '1' then                -- reset assíncrono tem prioridade
    cnt <= (others => '0');
  elsif rising_edge(clk) then      -- borda de subida do clock
    if en = '1' then
      if cnt = 9 then              -- ao chegar em 10, volta a zero
        cnt <= (others => '0');
      else
        cnt <= cnt + 1;
      end if;
    end if;
  end if;
end process p_contagem;
```

Como ler este trecho:

1. **Lista de sensibilidade** (clk, rst): o process só executa quando um desses sinais muda. Para gerar um registrador, ela contém o *clock* e o *reset* assíncrono.
2. `rst = '1'` primeiro: o reset é assíncrono, ou seja, zera `cnt` imediatamente, sem esperar o *clock*. Por isso ele vem antes de `rising_edge(clk)` no `if`.
3. `rising_edge(clk)`: detecta a transição de '0' para '1' do clock. Todas as atribuições dentro dessa condição geram elementos de armazenamento (flip-flops e registradores).
4. Contagem modulo 10: quando `cnt = 9`, o próximo valor é 0; nos demais casos, soma-se 1 ao valor atual.

Duas armadilhas comuns em processos:

- Atribuição a sinal (`<=`) não é imediata. O novo valor só aparece depois que o *process* termina de executar (no próximo *delta cycle*). Se você ler `cnt` logo depois de `cnt <= cnt + 1`, ainda obtém o valor antigo. Para um valor imediato, use uma variável com `:=`.
- Process combinacional incompleto. Se um `if` sem `else` dentro de um process combinacional deixar a saída sem valor em algum caso, o sintetizador cria um *latch*. Para evitar o latch, atribua um valor padrão no início do *process* ou utilize o *else*.

Para um reset síncrono, bastaria testar `rst` dentro do `elsif rising_edge(clk)` e tirar `rst` da lista de sensibilidade.

Design completo (design.vhd)

```
-- design.vhd : contador decimal (0 a 9) com reset, enable e saida de fim de contagem
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity contador_mod10 is
    port (
        clk : in  std_logic;           -- clock
        rst : in  std_logic;           -- reset assincrono, ativo em '1'
        en  : in  std_logic;           -- habilita a contagem
        q   : out std_logic_vector(3 downto 0); -- valor da contagem
        tc  : out std_logic;           -- '1' quando q = 9 (terminal count)
        seg : out std_logic_vector(6 downto 0) -- display 7 segmentos (gfedcba), ativo em '1'
    );
end entity contador_mod10;

architecture rtl of contador_mod10 is
    -- a contagem fica em um sinal interno: em VHDL-93 uma porta 'out' não pode ser lida
    signal cnt : unsigned(3 downto 0) := (others => '0');
begin

    -- Comandos SEQUENCIAIS (dentro do process): contador com registrador
    p_contagem : process (clk, rst)
    begin
        if rst = '1' then                -- reset assincrono tem prioridade
            cnt <= (others => '0');
        elsif rising_edge(clk) then      -- borda de subida do clock
            if en = '1' then
                if cnt = 9 then           -- ao chegar em 10, volta a zero
                    cnt <= (others => '0');
                else
                    cnt <= cnt + 1;
                end if;
            end if;
        end if;
    end process p_contagem;

    -- Comandos CONCORRENTES (fora de process): logica combinacional
    q <= std_logic_vector(cnt);          -- atribuicao simples
    tc <= '1' when cnt = 9 else '0';     -- atribuicao condicional (when/else)

    with cnt select
        seg <= "0111111" when "0000", -- 0
              "0000110" when "0001", -- 1
              "1011011" when "0010", -- 2
```

```

"1001111" when "0011", -- 3
"1100110" when "0100", -- 4
"1101101" when "0101", -- 5
"1111101" when "0110", -- 6
"0000111" when "0111", -- 7
"1111111" when "1000", -- 8
"1101111" when "1001", -- 9
"0000000" when others;

```

```
end architecture rtl;
```

7. Testbench: testando o circuito na simulação

O *testbench* é uma *entity* sem portas que instancia o circuito sob teste (DUT, *device under test*), gera o clock, aplica os estímulos e verifica as respostas. Como roda só no simulador, pode usar construções que não existem em hardware, como wait for 10 ns e assert.

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity tb_contador_mod10 is
    -- testbench nao tem portas
end entity tb_contador_mod10;

architecture sim of tb_contador_mod10 is

    constant T_CLK : time := 10 ns;                -- periodo do clock

    -- sinais que ligam o testbench ao DUT
    signal clk      : std_logic := '0';
    signal rst      : std_logic := '1';
    signal en       : std_logic := '0';
    signal q        : std_logic_vector(3 downto 0);
    signal tc       : std_logic;
    signal seg      : std_logic_vector(6 downto 0);

    signal fim_sim  : boolean := false;             -- controla o fim da simulacao

begin

    -- instanciacao do DUT com mapeamento por nome
    dut : entity work.contador_mod10
        port map (
            clk => clk,
            rst => rst,
            en  => en,
            q   => q,
            tc  => tc,
            seg => seg
        );

    -- geracao do clock (concorrente): inverte a cada meio periodo
    clk <= not clk after T_CLK / 2 when not fim_sim else clk;

```

```

-- estímulos (process sem lista de sensibilidade, usa wait)
p_estímulos : process
begin
    rst <= '1'; en <= '0';
    wait for 2 * T_CLK;          -- mantém reset por 2 ciclos
    rst <= '0';
    wait until rising_edge(clk);
    en <= '1';                  -- habilita a contagem
    wait for 25 * T_CLK;        -- passa por mais de 2 voltas completas

    en <= '0';                  -- pausa: q deve congelar
    wait for 3 * T_CLK;

    en <= '1';                  -- retoma
    wait for 4 * T_CLK;

    rst <= '1';                  -- reset assíncrono no meio da contagem
    wait for T_CLK / 2;
    rst <= '0';
    wait for 3 * T_CLK;

    fim_sim <= true;            -- para o clock, e a simulação termina
    wait;
end process p_estímulos;

-- verificação automática: q nunca pode passar de 9
p_verifica : process (clk)
begin
    if rising_edge(clk) then
        assert to_integer(unsigned(q)) <= 9
            report "ERRO: contagem acima de 9" severity failure;
    end if;
end process p_verifica;

end architecture sim;

```

O que cada parte faz:

- Entity vazia: o testbench e o topo da simulação e não tem conexão externa.
- Instanciação (entity work.contador_mod10 port map (...)): coloca o DUT dentro do *testbench* e liga cada porta a um sinal. O mapeamento por nome (clk => clk) evita erros de ordem.
- Clock: um comando concorrente que inverte clk a cada meia-período. A condição when not fim_sim para o clock no final, para que a simulação termine sozinha.
- Estímulos: um process sem lista de sensibilidade, que usa wait for e wait until para controlar o tempo. Ele termina com wait;, que suspende o process para sempre.
- Verificação: assert testa uma condição a cada borda de clock e, se ela for falsa, mostra a mensagem e interrompe a simulação (severity failure).

O comando *port map*: é o comando que realiza a instanciação de uma entidade vhdl. Como uma entidade pode ser instanciada diversas vezes, utilizam-se rótulos para diferenciá-las. Por exemplo, se instanciarmos dois contadores, podemos identificá-los pelos rótulos:

```
dut1 : entity work.contador_mod10
  port map (
    ...
  );

dut2 : entity work.contador_mod10
  port map (
    ...
  );
```

A instanciação de uma entity requer – como um chip em uma protoboard – a conexão das portas de entrada e saída em sinais, para comunicação entre o módulo e o ambiente externo.

A melhor maneira de realizar a conexão é identificando tanto a porta quanto o sinal pelo nome:

```
port map (
  clk => clk,      -- porta clk conectada ao sinal clk
  rst => rst,
  ...
```

8. Rodando a simulação e lendo a forma de onda

Clique em Run. Se o código compilar corretamente, o log mostra a execução e, com a opção marcada, o EPWave abre com as formas de onda; se houver erro de compilação, o log indica o arquivo e a linha. No EPWave, adicione os sinais clk, rst, en, q, tc e seg (use o botão de obter sinais se a lista não aparecer sozinha) e ajuste o zoom para ver os cerca de 380 ns da simulação.

9. Erros comuns, exercícios e resumo

Erros comuns no código e no ambiente EdaPlayground

- Esquecer *use IEEE.numeric_std.all*; faz o compilador não reconhecer *unsigned* nem *to_integer*.
- Ler uma porta out dentro da architecture (por exemplo, $q \leq q + 1$;) e erro em VHDL-93; use um sinal interno, como cnt.
- Um process sem lista de sensibilidade e sem *wait* roda em laço infinito e trava a simulação.
- Um *clock* que nunca para faz a simulação só terminar no limite de tempo do simulador; por isso o *testbench* usa fim_sim.
- Misturar *std_logic_vector* com numeros inteiros sem conversão (*unsigned(...)*, *std_logic_vector(...)*, *to_integer(...)*) gera erro de tipos.
- Se o simulador pedir a entity de topo e ela não for o testbench, a simulação roda o circuito errado; confira o campo Top entity.

Exercícios propostos

1. Troque o modulo 10 por um parametro: declare generic (MODULO : positive := 10) na *entity* e faça o contador reiniciar em MODULO - 1. Teste com 6 e com 16.
2. Torne o reset síncrono e compare, no EPWave, a resposta ao pulso de 5 ns do testbench.
3. Acrescente uma entrada *up* que escolha entre contagem crescente e decrescente (contador up/down que vai de 9 a 0).
4. Acrescente carga paralela: uma entrada *load* e um vetor *d* que substituem a contagem quando *load* = '1'.
5. Encadeie dois contadores modulo 10 usando *tc* como habilitação do segundo, para contar de 00 a 99. Use port map no design para instanciar os dois.

Resumo das construções

Construção	Onde aparece	Função
<i>entity</i>	inicio do design e do testbench	declara a interface dos módulos
<i>architecture</i>	depois da <i>entity</i>	descreve o seu comportamento
comando concorrente	corpo da <i>architecture</i>	logica combinacional sempre ativa; a ordem não importa
<i>process</i>	corpo da <i>architecture</i>	logica sequencial, executada em ordem; base dos registradores
port map	testbench e designs hierarquicos	instancia um componente e conecta suas portas aos sinais
<i>wait</i> , <i>assert</i>	process do testbench	controlam o tempo e verificam respostas (só p/ simulação)